Smart Crop AI viva guide built directly from the active Django repository.

Repository
C:\Users\AsadMirza\source\repos\Final Project - AI-Powered-Crop-Analysis\AI-Powered-Crop-Analysis\crop-ai-dashboard

Verification
Django test suite executed successfully on March 28, 2026 with 8 passing tests.

Contents

- How To Use This Guide
- 1. Project Foundation
- 2. Django Structure And Request Flow
- 3. Disease Detection Module
- 4. Yield Forecasting Module
- 5. Recommendation Center And Monitoring
- 6. Data, Models, And Training
- 6A. Model Training Deep Dive
- 7. Testing, Demo Support, And UI Logic
- 8. Important Files The Student Should Know
- 9. High-Value Viva Defense Lines
- 10. Things The Student Must Not Overclaim
- 11. Suggested Live Demo Flow For Viva
- 12. Rapid-Fire Viva Questions And Short Answers
- 13. Final Revision Sheet
- 14. Best Closing Line For Viva

Smart Crop AI Viva Preparation Guide

Repository: `C:\Users\AsadMirza\source\repos\Final Project - AI-Powered-Crop-Analysis\AI-Powered-Crop-Analysis\crop-ai-dashboard`

Generated on: March 28, 2026

Verification status:

- The current Django test suite was executed on March 28, 2026.
- Result: 8 tests passed.
- Command used: `python manage.py test`

Purpose of this guide:

- Help the student prepare for viva questions using the real repository, not a generic AI-agriculture explanation.
- Explain how each module works.
- Point to important files.
- Provide strong question-and-answer style responses for likely examiner questions.

How To Use This Guide

1. Read Sections 1 to 8 carefully.
2. Memorize the short defense lines in Sections 9 and 10.
3. Practice the demo flow in Section 11.
4. Use Section 12 as a rapid-revision sheet one day before viva.

1. Project Foundation

Q1. What is this project in one clear sentence?

Answer: This project is a local Django-based smart agriculture dashboard that combines crop disease detection, crop yield forecasting, and agronomic recommendations in one academic web application.

Key file references:

- `README.md`
- `docs/architecture.md`
- `smart_crop_ai/urls.py`

Q2. What problem is it trying to solve?

Answer: The project tries to support farmers or agriculture stakeholders by combining three practical decision-support functions:

- identify likely crop leaf diseases from images,
- forecast crop yield from environmental and soil inputs,
- generate irrigation and fertilizer recommendations from the forecast and crop profile.

This makes the system more useful than a single-model demo because it covers crop health, expected productivity, and actionable guidance.

Key file references:

- `README.md`
- `disease/views.py`
- `yield_prediction/views.py`
- `recommendations/views.py`

Q3. What is the active architecture of the repository?

Answer: The active implementation is a Django web application. The browser sends requests to Django, Django renders templates, and the backend calls either TensorFlow or scikit-learn services depending on the feature. The repository also stores prediction history in SQLite and exposes metrics, source, and recommendation pages.

Important clarification: The older `Next.js + FastAPI` prototype is not the active implementation in this repository anymore. If an examiner asks, the correct answer is that the current handover repository is Django-only.

Key file references:

- `docs/architecture.md`
- `smart_crop_ai/settings.py`
- `smart_crop_ai/urls.py`

Q4. Why was Django a sensible choice for this student project?

Answer: Django is a practical choice because it already provides:

- URL routing,
- forms and validation,
- template rendering,
- ORM models,
- admin panel,
- testing utilities,
- quick local deployment.

That means the student can show a complete working system without separately building frontend, authentication, database integration, and admin tooling from scratch.

Key file references:

- `smart_crop_ai/settings.py`
- `smart_crop_ai/urls.py`
- `disease/admin.py`
- `yield_prediction/admin.py`

Q5. What are the main functional modules in the codebase?

Answer: There are four main Django apps:

- `dashboard` : overview, history, metrics, and sources pages.
- `disease` : leaf image upload, TensorFlow disease inference, Grad-CAM overlay, and disease history storage.
- `yield_prediction` : form-based yield forecasting using a scikit-learn model and rule-linked agronomic outputs.
- `recommendations` : central page that combines the latest disease and yield recommendations.

Shared logic is placed under `smart_crop_ai/` for settings, reference data, demo samples, monitoring rules, and URL wiring.

Key file references:

- `dashboard/views.py`
- `disease/views.py`
- `yield_prediction/views.py`
- `recommendations/views.py`
- `smart_crop_ai/reference_data.py`
- `smart_crop_ai/demo_data.py`
- `smart_crop_ai/monitoring.py`

2. Django Structure And Request Flow

Q6. How are requests routed through the application?

Answer: The root URL configuration is in `smart_crop_ai/urls.py`. It maps:

- `/` to the dashboard app,
- `/disease/` to the disease app,
- `/yield/` to the yield prediction app,
- `/recommendations/` to the recommendation center,
- `/admin/` to Django admin.

This is a clean separation because each feature area has its own `urls.py` and `views.py`.

Key file references:

- `smart_crop_ai/urls.py`
- `dashboard/urls.py`
- `disease/urls.py`
- `yield_prediction/urls.py`
- `recommendations/urls.py`

Q7. How is runtime configuration handled?

Answer: Runtime configuration is handled in `smart_crop_ai/settings.py`. Important points are:

- `.env` is loaded using `python-dotenv`.
- `DEBUG`, `SECRET_KEY`, and `ALLOWED_HOSTS` are read from environment variables.
- SQLite is used as the default database.
- static and media directories are configured.
- model artifact paths are explicitly defined for disease and yield models.

This is good academic engineering because the paths for the shipped models are centralized in one place.

Key file references:

- `smart_crop_ai/settings.py`
- `.env`

Q8. How does the dashboard home page work?

Answer: The dashboard home page is rendered by `dashboard_home()` in `dashboard/views.py`, but most of the logic is actually built in `overview_context()` inside `dashboard/services.py`.

That service:

- fetches recent disease and yield records,
- computes averages and totals,
- builds alert counts,
- loads yield history for a spotlight crop,

- summarizes province coverage,
- prepares chart points for SVG display,
- sends operations notes to the template.

This is a good separation of concerns because the view stays small and the aggregation logic lives in a service layer.

Key file references:

- `dashboard/views.py`
- `dashboard/services.py`
- `templates/dashboard/home.html`

Q9. How does prediction history work?

Answer: Prediction history is not stored in text files. It is stored in the database using Django models:

- disease runs are stored in `DiseasePrediction`,
- yield runs are stored in `YieldPrediction`.

The history page queries the latest records, annotates their severity labels, and renders them in the dashboard. For yield records, it also builds a small line chart from the most recent saved forecasts.

Key file references:

- `dashboard/views.py`
- `disease/models.py`
- `yield_prediction/models.py`
- `templates/dashboard/history.html`

Q10. Why is the service layer important in this project?

Answer: The service layer keeps the code cleaner and easier to defend in viva:

- views handle request and response,
- services handle data preparation, ML inference, aggregation, and rule generation,
- models handle persistence.

This makes the architecture more maintainable than putting everything directly into Django views.

Key file references:

- `dashboard/services.py`
- `disease/services.py`
- `yield_prediction/services.py`
- `recommendations/services.py`

3. Disease Detection Module

Q11. What is the disease detection flow from user input to result?

Answer: The disease flow is:

1. User opens `/disease/`.
2. `DiseaseUploadForm` asks for crop and image, or accepts a built-in demo sample.
3. `disease_analysis()` validates the form.
4. If a demo sample is chosen, the file is loaded from `smart_crop_ai/demo_data.py`.
5. `get_disease_service()` returns a cached `DiseasePredictionService`.
6. `DiseasePredictionService.predict()` preprocesses the image, runs TensorFlow inference, filters probabilities for the selected crop, creates a Grad-CAM heatmap, computes leaf metrics, and saves an overlay image.
7. The result is stored in `DiseasePrediction`.
8. The template shows the uploaded image, overlay image, confidence distribution, and recommendations.

Key file references:

- `disease/forms.py`
- `disease/views.py`
- `disease/services.py`
- `disease/models.py`
- `smart_crop_ai/demo_data.py`
- `templates/disease/analyze.html`

Q12. How does form validation work in the disease module?

Answer: The form allows either:

- a real uploaded image, or
- a built-in demo sample.

The custom `clean()` method in `DiseaseUploadForm` raises a validation error if neither is provided. This avoids empty inference requests.

Key file references:

- `disease/forms.py`

Q13. What model is used for disease detection?

Answer: The shipped disease classifier uses TensorFlow with a MobileNetV2-based transfer learning pipeline. According to the metadata, the model family is `MobileNetV2`.

This is a sensible choice because MobileNetV2 is lighter than many larger CNNs and is suitable for academic demos where inference must still run locally.

Key file references:

- `models/disease/smart_crop_disease_metadata.json`
- `training/disease/train_demo_disease_model.py`

Q14. How is the uploaded image processed before prediction?

Answer: Inside `DiseasePredictionService`:

- raw bytes are decoded with OpenCV,
- the image is converted from BGR to RGB,
- it is resized to the configured image size from metadata,
- pixel values are normalized to the `0-1` range,
- a batch dimension is added before TensorFlow prediction.

This is the standard preprocessing step needed before passing the image into the CNN.

Key file references:

- `disease/services.py`

Q15. What does crop-aware filtering mean in this project?

Answer: The disease model was trained on multiple crops and multiple classes, but the user also tells the system which crop the leaf belongs to. After the model outputs class probabilities, the service masks out classes that do not belong to that selected crop, then renormalizes the remaining probabilities.

Why this matters:

- It reduces obviously irrelevant predictions from unrelated crops.
- It makes the diagnosis more practical for a crop-specific workflow.

Important limitation: If the user selects the wrong crop, the prediction is constrained to the wrong crop family. So crop-aware filtering improves focus, but it also depends on correct crop selection.

Key file references:

- `disease/services.py`
- `smart_crop_ai/reference_data.py`
- `models/disease/smart_crop_disease_metadata.json`

Q16. How is explainability shown for disease predictions?

Answer: The project uses a Grad-CAM style heatmap. The service builds a model that outputs the final convolution layer and the prediction layer, computes gradients for the predicted class, converts that into a heatmap, and overlays the heatmap on the original image with OpenCV.

The UI calls this an `OpenCV + Grad-CAM overlay`.

This gives the student a strong viva point: the project does not only output a class name, it also shows where the model focused in the image.

Key file references:

- `disease/services.py`
- `templates/disease/analyze.html`

Q17. What extra metrics are computed in the disease module beyond class prediction?

Answer: Two extra percentages are computed:

- `plant_coverage_pct` : how much of the image seems to be leaf area,
- `hotspot_pct` : how much of the visible leaf area falls inside the high-activation stress region.

These are used in the result summary and also influence alert severity.

Key file references:

- `disease/services.py`
- `smart_crop_ai/monitoring.py`

Q18. How are disease recommendations generated?

Answer: Disease recommendations are generated by a rule-based function, not by the TensorFlow model directly. The function `build_disease_recommendations()` maps disease labels like `Late Blight` , `Common Rust` , or `Tomato Yellow Leaf Curl Virus` to guidance text.

Then it adds a final note depending on confidence and hotspot level:

- strong case if confidence is high and hotspot is pronounced,
- softer scouting guidance if confidence is moderate.

This hybrid design is important: AI is used for classification, while recommendations are rule-based and easier to explain.

Key file references:

- `recommendations/services.py`
- `disease/services.py`

Q19. What is stored in the disease database table?

Answer: The `DiseasePrediction` model stores:

- crop,
- original image,
- overlay image,
- predicted label,
- confidence,
- summary,
- probabilities as JSON,
- recommendations as JSON,
- plant coverage percentage,
- hotspot percentage,
- creation timestamp.

Using `JSONField` is practical here because probabilities and recommendation lists have variable length and do not need separate normalized tables for this academic scope.

Key file references:

- `disease/models.py`
- `disease/migrations/0001_initial.py`

Q20. What are the main strengths and limitations of the disease module?

Answer: Strengths:

- multi-crop coverage,
- real public datasets,
- explainability overlay,
- demo samples for reproducibility,
- persisted prediction history.

Limitations:

- it is still a classification model trained mainly on public datasets, not a clinically certified agronomy system,
- real-field generalization is limited by the gap between cleaner datasets and uncontrolled farm imagery,
- crop selection by user affects crop-aware filtering.

Key file references:

- `docs/datasets.md`
- `docs/architecture.md`
- `models/disease/smart_crop_disease_metadata.json`

4. Yield Forecasting Module

Q21. What is the yield prediction flow from form to output?

Answer: The yield flow is:

1. User opens `/yield/`.
2. The form is prefilled with a default demo payload.
3. The user chooses crop, district, season, year, rainfall, temperature, soil values, nutrients, and cultivated area.
4. `yield_forecast()` validates the form and calls `get_yield_service()`.
5. `YieldPredictionService.predict()` builds the feature frame required by the trained scikit-learn pipeline.
6. The service predicts yield in `t/ha`.
7. It compares the result with the historical baseline.
8. It generates top factors and agronomic recommendations.
9. The result is saved in `YieldPrediction`.
10. The page renders the forecast, historical chart, and recommendation cards.

Key file references:

- `yield_prediction/forms.py`
- `yield_prediction/views.py`
- `yield_prediction/services.py`
- `yield_prediction/models.py`
- `templates/yield_prediction/forecast.html`

Q22. What model is used for yield forecasting?

Answer: The active yield model is a scikit-learn `RandomForestRegressor`. The metadata shows that it was selected as the best model among:

- linear regression,
- random forest,
- histogram gradient boosting.

It won because it achieved the best evaluation metrics in the shipped metadata.

Key file references:

- `models/yield/smart_crop_yield_metadata.json`
- `training/yield/train_yield_model.py`

Q23. Why is scikit-learn more suitable than deep learning for the yield task here?

Answer: Yield prediction in this repository is based on structured tabular data, not images. For tabular data of this scale, tree-based models are often more practical than deep neural networks because they:

- train faster,

- require less tuning,
- work well on mixed numerical and encoded categorical features,
- are easier to compare in a small academic project.

That is why the student can defend scikit-learn as a more appropriate engineering choice for this dataset.

Key file references:

- `training/yield/train_yield_model.py`
- `models/yield/smart_crop_yield_metadata.json`

Q24. What features does the yield model actually use?

Answer: The shipped model uses these feature columns:

- `crop`
- `year`
- `rainfall_mm`
- `avg_temp_c`
- `soil_ph`
- `organic_carbon_pct`
- `soil_nitrogen_ppm`
- `cultivated_area_hectares`
- `pbs_crops_output_constant_rs_mn`
- `pbs_important_crops_output_constant_rs_mn`

Very important viva detail: The form collects `nitrogen`, `phosphorus`, and `potassium`, but the trained model directly uses nitrogen only. Phosphorus and potassium are mainly used in the rule-based recommendation layer, not in the current trained ML feature list.

That is a subtle but correct answer if the examiner asks, "Do all form inputs go into the model?"

Key file references:

- `yield_prediction/services.py`
- `yield_prediction/forms.py`
- `models/yield/smart_crop_yield_metadata.json`

Q25. How is the feature frame built at runtime?

Answer: The service reads the model's expected feature columns from metadata and constructs a one-row pandas DataFrame. If the model expects `soil_nitrogen_ppm`, the service maps it from the form field `nitrogen_ppm`.

For annual macro features, the service also fills PBS-based economic context values using the nearest available year from metadata. This makes the runtime logic consistent with how the training dataset was assembled.

Key file references:

- `yield_prediction/services.py`
- `models/yield/smart_crop_yield_metadata.json`

Q26. What historical baseline is used to compare predicted yield?

Answer: The current shipped metadata uses `Pakistan historical average`, not district-labeled ground-truth yield history. The service compares the predicted yield against that national historical average for the selected crop.

This is one of the most important honesty points in viva:

- district is used as operational context for weather defaults, soil defaults, and recommendation context,
- but the historical yield comparison is still Pakistan-level in the shipped artifacts.

Key file references:

- `models/yield/smart_crop_yield_metadata.json`
- `yield_prediction/services.py`
- `docs/datasets.md`
- `docs/architecture.md`

Q27. Why does the code still accept an optional district argument for history?

Answer: The service methods `history_for()` and `_historical_average()` are written in a future-ready way. If metadata later becomes district-specific, the service can use nested structures. Right now, because the metadata stores crop-level lists, the district argument is effectively ignored and the crop-level history is returned.

This is a good answer if the examiner asks whether the code was designed for extension.

Key file references:

- `yield_prediction/services.py`
- `models/yield/smart_crop_yield_metadata.json`

Q28. How are top factors shown for the yield result?

Answer: Top factors are generated by `_top_factors()` in the yield service. They are not SHAP values or model-native feature importances. They are rule-derived interpretation items comparing input values with crop comfort targets, such as:

- rainfall positioning,
- temperature stress,
- nitrogen status,
- organic carbon buffer.

This means the project offers explainability-like summaries without claiming they are formal causal proofs.

Key file references:

- `yield_prediction/services.py`
- `smart_crop_ai/reference_data.py`

Q29. How are agronomic recommendations generated?

Answer: Agronomic recommendations are produced by `build_agronomic_recommendations()` in `recommendations/services.py`. The logic compares the user inputs against crop targets and forecast performance.

Examples:

- low rainfall or high temperature can trigger irrigation advice,
- low nitrogen can trigger high-priority fertilizer advice,
- low phosphorus or potassium can trigger medium-priority nutrient advice,
- low organic carbon can trigger organic matter recommendations,
- below-average forecast can trigger investigation guidance.

This is a rule-based recommendation engine attached to an ML forecast, not a second AI model.

Key file references:

- `recommendations/services.py`
- `yield_prediction/services.py`
- `smart_crop_ai/reference_data.py`

Q30. What is stored in the yield database table?

Answer: The `YieldPrediction` model stores:

- crop,
- district,
- province,
- season,
- year,
- rainfall,
- temperature,
- soil values,
- NPK inputs,
- cultivated area,
- predicted yield,
- historical average,
- delta versus average,
- summary,
- top factors as JSON,
- recommendations as JSON,
- historical series as JSON,
- timestamp.

This gives the dashboard enough information to show both numerical history and decision support without recomputing everything on every page load.

Key file references:

- `yield_prediction/models.py`
- `yield_prediction/migrations/0001_initial.py`

Q31. What are the main strengths and limitations of the yield module?

Answer: Strengths:

- uses real public data sources,
- compares multiple classical ML models,
- stores full forecast history,
- includes smart defaults and demo scenarios,
- separates ML forecast from recommendation logic.

Limitations:

- historical comparison is Pakistan-level, not district-labeled yield ground truth,
- phosphorus and potassium are not direct model features in the shipped model,
- recommendations are rule-based, not expert-verified agronomy prescriptions.

Key file references:

- `docs/datasets.md`
- `training/yield/train_yield_model.py`
- `models/yield/smart_crop_yield_metadata.json`

5. Recommendation Center And Monitoring

Q32. What is the recommendation center?

Answer: The recommendation center is a separate page that collects the latest disease and yield outputs into one decision layer. It shows:

- the latest yield-linked agronomic recommendations,
- the latest disease guidance,
- recent yield items,
- recent disease items,
- alert counts and category counts.

This makes the app feel like a decision-support dashboard rather than two isolated models.

Key file references:

- `recommendations/views.py`
- `templates/recommendations/center.html`

Q33. How is alert severity computed?

Answer: Alert severity is computed by simple monitoring rules:

- disease severity is high if the label is not healthy and either confidence is high or hotspot percentage is high,
- yield severity depends on recommendation priorities and the absolute forecast deviation from average.

The labels are:

- `high` -> `High Alert`
- `medium` -> `Watch`
- `low` -> `Stable`

This is deliberately simple and explainable logic.

Key file references:

- `smart_crop_ai/monitoring.py`
- `dashboard/services.py`

Q34. Why separate AI inference from recommendation and alert logic?

Answer: This is one of the best design choices to defend in viva. The project separates:

- AI/ML tasks: disease classification and yield regression,
- rule-based tasks: recommendations, alert severity, and operational guidance.

Reasons:

- easier to explain,

- easier to test,
- easier to adjust without retraining models,
- safer for an academic system where recommendations should stay auditable.

Key file references:

- `disease/services.py`
- `yield_prediction/services.py`
- `recommendations/services.py`
- `smart_crop_ai/monitoring.py`

Q35. What do the metrics and sources pages add to the project?

Answer: They improve academic defensibility. Instead of only showing predictions, the app also shows:

- model metrics,
- dataset scale,
- supported crops and districts,
- candidate model comparison,
- data source descriptions,
- limitations and runtime notes.

This helps the student answer examiner questions using pages already built into the software.

Key file references:

- `dashboard/services.py`
- `templates/dashboard/metrics.html`
- `templates/dashboard/sources.html`
- `models/disease/smart_crop_disease_metadata.json`
- `models/yield/smart_crop_yield_metadata.json`

6. Data, Models, And Training

Q36. Which datasets are used in the disease pipeline?

Answer: The disease pipeline uses public datasets from:

- PlantVillage,
- PlantDoc,
- crop-specific Mendeley datasets for cotton and sugarcane.

This combination is important because:

- PlantVillage gives a stable baseline,
- PlantDoc adds more field-like images,
- Mendeley expands crop coverage beyond the standard PlantVillage subset.

Key file references:

- `docs/datasets.md`
- `training/disease/train_demo_disease_model.py`
- `models/disease/smart_crop_disease_metadata.json`

Q37. Which datasets are used in the yield pipeline?

Answer: The yield pipeline uses:

- FAOSTAT for crop statistics,
- Pakistan Bureau of Statistics for macro-agriculture economic context,
- NASA POWER for weather features,
- SoilGrids for soil features.

This is a strong answer because it shows the project is not based on random synthetic numbers in the current version.

Key file references:

- `docs/datasets.md`
- `training/yield/train_yield_model.py`
- `models/yield/smart_crop_yield_metadata.json`

Q38. How is the disease training pipeline built?

Answer: The training script:

- ensures data directories exist,
- loads PlantVillage images,
- downloads selected PlantDoc images,
- downloads selected Mendeley archives,
- preprocesses images with OpenCV,
- creates a manifest CSV,

- builds a combined dataset,
- splits it into train, validation, and test sets,
- trains a MobileNetV2 transfer-learning model,
- fine-tunes the last part of the base network,
- saves the `.keras` model and metadata JSON.

Good viva detail: The split is done using `train_test_split` with stratification, which is a reasonable method for balanced academic evaluation.

Key file references:

- `training/disease/train_demo_disease_model.py`
- `data/processed/disease/disease_dataset_manifest.csv`
- `models/disease/smart_crop_disease.keras`
- `models/disease/smart_crop_disease_metadata.json`

Q39. How is the yield training pipeline built?

Answer: The yield training script:

- downloads or refreshes official source files,
- extracts FAOSTAT crop panel data,
- parses PBS annual accounts,
- fetches NASA POWER monthly climate data,
- samples SoilGrids properties around reference districts,
- merges everything into one processed CSV,
- compares three candidate regressors,
- selects the best one by evaluation metrics,
- saves the trained joblib pipeline and metadata JSON.

This is strong to mention because it shows the project includes a reproducible data-engineering path, not just a hard-coded model file.

Key file references:

- `training/yield/train_yield_model.py`
- `data/processed/pakistan_multi_crop_yield_dataset.csv`
- `models/yield/smart_crop_yield_model.joblib`
- `models/yield/smart_crop_yield_metadata.json`

Q40. What are the shipped model performance numbers?

Answer: Disease model:

- model family: `MobileNetV2`
- validation accuracy after fine-tuning: `81.04%`
- test accuracy: `80.60%`
- image count used: `5768`
- classes covered: `24`

Yield model:

- best model: `Random Forest`
- RMSE: `1.9055`
- MAE: `1.1597`
- R2: `0.9949`
- feature count: `10`
- history span: `2001-2024`

These numbers are already readable from the app's metrics page because the page is populated from metadata files.

Key file references:

- `models/disease/smart_crop_disease_metadata.json`
- `models/yield/smart_crop_yield_metadata.json`
- `templates/dashboard/metrics.html`

Q41. What model artifacts are already included in the repository?

Answer: The repository already includes:

- `models/disease/smart_crop_disease.keras`
- `models/disease/smart_crop_disease_metadata.json`
- `models/yield/smart_crop_yield_model.joblib`
- `models/yield/smart_crop_yield_metadata.json`
- `data/processed/disease/disease_dataset_manifest.csv`
- `data/processed/pakistan_multi_crop_yield_dataset.csv`

That means normal project use does not require retraining.

Key file references:

- `README.md`
- `docs/datasets.md`

Q42. What dependencies are needed for runtime and for retraining?

Answer: For normal runtime, `requirements.txt` is enough. It includes Django, TensorFlow CPU, OpenCV, pandas, NumPy, Pillow, scikit-learn, joblib, and dotenv support.

For retraining, `requirements-training.txt` adds extra packages such as:

- `openpyxl`
- `requests`
- `rioxarray`
- `soilgrids`

This separation is good because it keeps normal app setup lighter than the training environment.

Key file references:

- `requirements.txt`
- `requirements-training.txt`

- `README.md`

6A. Model Training Deep Dive

Q42A. What exact default training settings are used for the disease model?

Answer: The default disease training configuration in `training/disease/train_demo_disease_model.py` is:

- image size: `160 x 160`
- maximum PlantVillage images per class: `220`
- maximum PlantDoc images per class: `55`
- maximum Mendeley images per class: `220`
- head training epochs: `3`
- fine-tuning epochs: `2`
- batch size: `24`
- initial optimizer: Adam with learning rate `1e-3`
- fine-tuning optimizer: Adam with learning rate `1e-4`
- early stopping: monitor `val_accuracy`, patience `2`, restore best weights

The augmentation pipeline applies:

- horizontal flip,
- small rotation of `0.05`,
- small zoom of `0.08`.

Important viva point: These are modest settings chosen for a practical local academic project, not a large compute-intensive experiment.

Key file references:

- `training/disease/train_demo_disease_model.py`

Q42B. How is the disease dataset split, and why is that reasonable?

Answer: The disease dataset is split in two stages:

- first split: `70%` train and `30%` temporary set,
- second split: the temporary set is divided equally into validation and test,
- final result: `70%` train, `15%` validation, `15%` test.

The split is stratified using class indices. That means class balance is preserved across train, validation, and test sets.

Why this is reasonable:

- train set is used for fitting,
- validation set is used for early stopping and tuning stage control,
- test set is kept for final evaluation,
- stratification avoids distorted results from class imbalance.

Key file references:

- `training/disease/train_demo_disease_model.py`

Q42C. What exactly happens during disease fine-tuning?

Answer: The disease model starts with MobileNetV2 as a pretrained backbone with ImageNet weights. In the first stage, the backbone is frozen and only the new task-specific layers are trained. In the second stage, the backbone is unfrozen partially:

- all layers except the last `30` remain frozen,
- batch normalization layers are kept frozen,
- the model is trained again with a lower learning rate.

This is a standard transfer-learning strategy because:

- the pretrained model already contains useful visual features,
- the custom head learns crop-disease classes,
- partial unfreezing lets the network adapt without destabilizing all pretrained weights.

Key file references:

- `training/disease/train_demo_disease_model.py`

Q42D. What exact training design is used for the yield model?

Answer: The yield training pipeline in `training/yield/train_yield_model.py` does the following:

- builds one processed tabular dataset from FAOSTAT, PBS, NASA POWER, and SoilGrids,
- defines `10` feature columns,
- one-hot encodes the categorical `crop` field,
- passes numerical fields through unchanged,
- compares three candidate models:
- `LinearRegression()`
- `RandomForestRegressor(n_estimators=320, random_state=42, max_depth=12)`
- `HistGradientBoostingRegressor(random_state=42, max_depth=6)`

The best pipeline is saved as:

- `models/yield/smart_crop_yield_model.joblib`

Important viva point: This is a classical tabular ML pipeline with preprocessing and model combined into one saved scikit-learn pipeline object.

Key file references:

- `training/yield/train_yield_model.py`
- `models/yield/smart_crop_yield_model.joblib`

Q42E. How is the yield dataset split, and why is that stronger than a random split here?

Answer: The yield dataset is split by time, not randomly:

- training data: years `<= max_year - 3`
- test data: years `> max_year - 3`

Given the current shipped dataset span of `2001-2024`, that means:

- train period: `2001-2021`
- test period: `2022-2024`

Why this is stronger:

- it better reflects forecasting by testing on future years,
- it reduces temporal leakage,
- it is more realistic than randomly mixing earlier and later years together.

Important honesty point: The script does not currently use cross-validation for the yield candidates. It uses a time-based holdout test, which is reasonable for this project but still not the strongest possible experimental design.

Key file references:

- `training/yield/train_yield_model.py`
- `models/yield/smart_crop_yield_metadata.json`

Q42F. What feature engineering and preprocessing happen in the yield training pipeline?

Answer: The yield pipeline performs several preprocessing steps:

- FAOSTAT crop data is filtered to Pakistan, target crops, and years from `2001` onward.
- FAOSTAT `yield_kg_ha` is converted into `yield_t_ha`.
- PBS annual account columns are parsed into usable numeric years.
- NASA POWER monthly data is aggregated into seasonal rainfall and average temperature.
- SoilGrids raster values are sampled and averaged into soil features.
- district-level reference values are averaged across crop-specific reference districts.
- all data sources are merged into one processed dataset.
- missing rows are dropped before model training.
- `crop` is one-hot encoded using `OneHotEncoder`.

This is important to explain because the intelligence of the yield model is not only in the regressor. It is also in how the multi-source dataset is assembled.

Key file references:

- `training/yield/train_yield_model.py`
- `data/processed/pakistan_multi_crop_yield_dataset.csv`

Q42G. If the examiner asks, "How do you retrain the models?", what should the student answer?

Answer: The correct retraining sequence is:

`pip install -r requirements.txt`

1. Install runtime dependencies.

```
pip install -r requirements-training.txt
```

1. Install training extras.

```
python training\\disease\\train_demo_disease_model.py
```

1. Retrain the disease model if needed.

```
python training\\yield\\train_yield_model.py --refresh-sources
```

1. Retrain the yield model if needed.

Important clarification: Retraining is optional. The repository already ships with trained artifacts, so normal project use does not require these steps.

Key file references:

- `README.md`
- `training/disease/train_demo_disease_model.py`
- `training/yield/train_yield_model.py`

Q42H. What are the strongest honest limitations in the training design?

Answer: Disease training limitations:

- public datasets still create a lab-to-field or curated-to-real-world gap,
- crop-aware filtering helps inference, but wrong crop selection can still constrain predictions badly,
- the current setup is a good academic baseline, not a medical-grade or agronomy-certified diagnostic system.

Yield training limitations:

- historical comparison remains Pakistan-level rather than district-labeled ground truth,
- the dataset is relatively small compared with large industrial forecasting systems,
- there is no cross-validation or uncertainty estimation in the shipped training script,
- phosphorus and potassium are not direct ML features in the active model.

This is exactly the kind of honest answer that usually strengthens a viva rather than weakening it.

Key file references:

- `docs/datasets.md`
- `docs/architecture.md`
- `training/disease/train_demo_disease_model.py`
- `training/yield/train_yield_model.py`

Q42I. What future training improvements would be strongest if the project were extended?

Answer: The highest-value next improvements would be:

- collect more real-field Pakistani disease images for better domain adaptation,
- obtain district-labeled historical yield targets instead of relying on Pakistan-level averages,

- add vegetation-index features like `NDVI` or `EVI`,
- use stronger evaluation such as time-series cross-validation for yield,
- tune more hyperparameters systematically,
- add uncertainty estimates or calibrated confidence,
- expand ablation studies to show which features matter most.

This is a strong viva answer because it shows both technical understanding and awareness of how the system could mature beyond the current academic scope.

Key file references:

- `docs/datasets.md`
- `training/disease/train_demo_disease_model.py`
- `training/yield/train_yield_model.py`

7. Testing, Demo Support, And UI Logic

Q43. What automated tests exist in the project?

Answer: The repository includes tests for:

- dashboard pages,
- disease upload flow,
- disease demo sample flow,
- disease page crop visibility,
- yield page demo controls,
- yield forecast database creation,
- yield historical range display,
- recommendation center route availability.

This is not exhaustive enterprise testing, but it is enough to show that the main routes and critical flows were validated.

Key file references:

- `dashboard/tests.py`
- `disease/tests.py`
- `yield_prediction/tests.py`
- `recommendations/tests.py`

Q44. How do the demo features help the viva?

Answer: The project includes built-in demo support for both major AI flows:

- disease demo images such as maize rust, potato late blight, cotton bacterial blight, and sugarcane yellow leaf,
- yield demo scenarios such as balanced maize, heat-stress wheat, and high-potential sugarcane.

This helps the student present a stable live demo without relying on random external input during viva.

Key file references:

- `smart_crop_ai/demo_data.py`
- `templates/disease/analyze.html`
- `templates/yield_prediction/forecast.html`

Q45. How does district autofill work in the yield page?

Answer: The backend passes district baseline values and crop defaults into the template as JSON. JavaScript in `forecast.html` listens for district and crop changes and fills fields such as:

- rainfall,
- temperature,
- soil pH,

- organic carbon,
- season,
- nutrient targets.

This is a UI productivity feature, not part of the ML model itself.

Key file references:

- `yield_prediction/views.py`
- `templates/yield_prediction/forecast.html`
- `smart_crop_ai/reference_data.py`

Q46. Why is `lru_cache` used for the model services?

Answer: Both `get_disease_service()` and `get_yield_service()` are wrapped with `@lru_cache(maxsize=1)`. This means the heavy model objects are loaded once and then reused, instead of reloading TensorFlow and joblib artifacts on every request.

This improves local performance and is a smart optimization to mention in viva.

Key file references:

- `disease/services.py`
- `yield_prediction/services.py`

8. Important Files The Student Should Know

If the examiner asks, "Where in code is that implemented?", these are the best files to mention quickly:

Runtime configuration, SQLite setup, model artifact paths, static and media paths.

- `smart_crop_ai/settings.py`

Top-level route wiring.

- `smart_crop_ai/urls.py`

Crop profiles, district profiles, and disease class definitions.

- `smart_crop_ai/reference_data.py`

Built-in demo scenarios for viva-safe testing.

- `smart_crop_ai/demo_data.py`

Alert severity rules and labels.

- `smart_crop_ai/monitoring.py`

Dashboard aggregation logic, metrics loading, sources page context, chart point generation.

- `dashboard/services.py`

Validates that either an image or demo sample is provided.

- `disease/forms.py`

Main disease request handling and database save.

- `disease/views.py`

TensorFlow inference, preprocessing, crop-aware filtering, Grad-CAM, overlay saving.

- `disease/services.py`

Disease prediction schema in SQLite.

- `disease/models.py`

Forecast input schema for the yield model.

- `yield_prediction/forms.py`

Yield forecast request handling and result persistence.

- `yield_prediction/views.py`

Feature-frame construction, scikit-learn prediction, top factors, historical comparison.

- `yield_prediction/services.py`

Yield prediction schema in SQLite.

- `yield_prediction/models.py`

Rule-based agronomic and disease recommendation logic.

- `recommendations/services.py`

Combined recommendation center.

- `recommendations/views.py`

Disease dataset assembly and TensorFlow training pipeline.

- `training/disease/train_demo_disease_model.py`

Yield dataset assembly and scikit-learn training pipeline.

- `training/yield/train_yield_model.py`

Disease metrics, class counts, and model configuration.

- `models/disease/smart_crop_disease_metadata.json`

Yield model choice, metrics, history, and feature columns.

- `models/yield/smart_crop_yield_metadata.json`

9. High-Value Viva Defense Lines

These are short, strong statements the student can say with confidence:

- "The active repository is a Django web application; the earlier Next.js and FastAPI prototype is not the current runtime."
- "The project uses a hybrid design: AI for prediction, rule-based logic for recommendations and alerting."
- "Disease detection uses TensorFlow MobileNetV2 transfer learning with Grad-CAM-style explainability."
- "Yield forecasting uses a scikit-learn Random Forest trained on tabular data, which is more suitable than a CNN for this problem."
- "SQLite was chosen for local handover simplicity and zero-setup demonstration."
- "The current historical yield comparison is Pakistan-level, while district is used as operational context for weather, soil defaults, and recommendations."
- "Not every form input is a direct ML feature; phosphorus and potassium mostly feed the rule-based recommendation layer in the shipped version."
- "The app stores prediction history so the dashboard can act like a monitoring board, not just a single prediction form."

10. Things The Student Must Not Overclaim

These points are important. Saying the wrong thing here can weaken the viva answer:

- Do not say the yield baseline is district ground truth. The shipped metadata says `Pakistan historical average`.
- Do not say every agronomic recommendation comes from AI. The recommendations are rule-based.
- Do not say the yield model uses all NPK values directly. The current feature list directly uses nitrogen, not phosphorus and potassium.
- Do not say the system is deployed as a cloud production platform. The repository is designed for local web deployment.
- Do not say the disease model was trained only on Pakistani farm images. It uses public datasets like PlantVillage, PlantDoc, and Mendeley crop disease sets.
- Do not say the project has multi-user production scaling. SQLite and the current setup target local academic usage.

11. Suggested Live Demo Flow For Viva

Q47. How should the student present the demo in 5 to 7 minutes?

Answer: Use this order:

Explain that the dashboard combines monitoring, recent predictions, alert counts, yield history, and evidence access.

1. Start at `/`.

Show that the project reports real model metrics, dataset counts, and supported crop coverage.

1. Open `/metrics/`.

Explain the free public data sources and state the main limitation honestly.

1. Open `/sources/`.

Run one built-in sample like maize rust or potato late blight. Explain:

1. Open `/disease/`.

- upload or demo mode,
- TensorFlow classification,
- crop-aware filtering,
- overlay image,
- recommendations,
- saved history.

Load a built-in scenario like balanced maize or heat-stress wheat. Explain:

1. Open `/yield/`.

- district autofill,
- tabular features,
- Random Forest prediction,
- comparison with historical average,
- top factors,
- agronomic recommendations.

Show that the system combines the latest disease and yield guidance into one monitoring surface.

1. Open `/recommendations/`.

Show stored records and mention SQLite persistence.

1. Open `/history/`.

Key file references:

- `templates/dashboard/home.html`

- `templates/dashboard/metrics.html`
- `templates/dashboard/sources.html`
- `templates/disease/analyze.html`
- `templates/yield_prediction/forecast.html`
- `templates/recommendations/center.html`
- `templates/dashboard/history.html`

12. Rapid-Fire Viva Questions And Short Answers

Q48. What database is used?

Answer: SQLite, configured in `smart_crop_ai/settings.py`.

Q49. Why SQLite?

Answer: Because this repository is intended for zero-setup local handover and viva demonstration, not production-scale multi-user deployment.

Q50. What framework powers the web app?

Answer: Django.

Q51. Which part uses TensorFlow?

Answer: The disease detection module.

Q52. Which part uses scikit-learn?

Answer: The yield forecasting module.

Q53. Is the recommendation center itself an AI model?

Answer: No. It aggregates outputs and rule-based guidance from other parts of the system.

Q54. How are disease results made more explainable?

Answer: By generating an OpenCV plus Grad-CAM overlay that highlights model attention areas.

Q55. What does the disease model predict?

Answer: A crop-specific disease label with probabilities and confidence.

Q56. How many disease classes are in the shipped metadata?

Answer: 24 classes.

Q57. Which crops are supported for disease detection?

Answer: Maize, potato, tomato, cotton, and sugarcane.

Q58. Which crops are supported for yield forecasting?

Answer: Maize, wheat, rice, cotton, and sugarcane.

Q59. What is the best shipped yield model?

Answer: Random Forest.

Q60. What does RMSE mean?

Answer: Root Mean Squared Error. It measures average prediction error magnitude with more penalty for larger errors.

Q61. What does R2 mean?

Answer: It measures how much of the variance in the target is explained by the model.

Q62. Where are trained models stored?

Answer: Under `models/disease/` and `models/yield/`.

Q63. Where is prediction history stored?

Answer: In SQLite through Django models `DiseasePrediction` and `YieldPrediction`.

Q64. What is the role of JSONField in this project?

Answer: It stores flexible structured outputs like probabilities, recommendations, top factors, and historical series.

Q65. Why does the yield page ask for district if the history baseline is national?

Answer: District still matters for operational context, local defaults, weather and soil assumptions, province labeling, and recommendation relevance.

Q66. How are demo samples useful?

Answer: They make the system reproducible and viva-safe because the student can demonstrate known working cases quickly.

Q67. Is the project retrained every time it runs?

Answer: No. The repository ships with trained artifacts. Retraining is optional.

Q68. What is one honest limitation to mention confidently?

Answer: The historical yield comparison is still Pakistan-level instead of district-labeled ground truth.

Q69. What is another honest limitation?

Answer: Agronomic recommendations are rule-based decision support, not expert-certified prescriptions.

Q70. What makes the project more than a simple ML notebook?

Answer: It is an integrated web application with forms, persistence, dashboards, metrics pages, demo mode, recommendations, and tests.

13. Final Revision Sheet

If the student has only 2 minutes before viva, remember these points:

- The project is a Django smart agriculture dashboard.
- It has three major user-facing functions: disease detection, yield forecasting, and recommendations.
- Disease detection uses TensorFlow MobileNetV2 and Grad-CAM-style explainability.
- Yield forecasting uses a scikit-learn Random Forest on tabular data.
- Recommendations and alert levels are rule-based.
- Data sources are public: PlantVillage, PlantDoc, Mendeley, FAOSTAT, PBS, NASA POWER, and SoilGrids.
- Database is SQLite.
- Models are already included in the repository.
- The major limitation is that yield history comparison is Pakistan-level, not district ground truth.
- The student should never claim that all recommendations come from AI.

14. Best Closing Line For Viva

If the examiner asks for a short conclusion, the student can say:

"This project is a complete local decision-support dashboard for crop analysis. Its strength is not only the use of AI models, but the integration of prediction, explainability, recommendations, persistence, and evidence pages in one Django system. At the same time, I clearly recognize its current limitations, especially the Pakistan-level yield baseline and the rule-based nature of recommendation logic."